

Phase 2.5 — Practical Assignments (Week 2)

Week 2 — Processes & Signals

🎯 Assignment 1: Process Investigation

1. Start multiple background processes:

- `sleep`
- `yes > /dev/null`

2. Use:

- `ps`
- `top`
- `pstree`
- `lsof`

Solutions: -

1. **Background processes:**

- Sometimes we want a program to run **in the background** so we can keep using the terminal.

- (a) **Sleep**

```
garima@garima-HP-Pavilion-Notebook:~$ sleep 100 &
[1] 5377
garima@garima-HP-Pavilion-Notebook:~$ echo "hello"
hello
```

This command will wait for 100 sec. & will make it run in background. We can still use the terminal

- (b) **yes > /dev/null**

```
garima@garima-HP-Pavilion-Notebook:~$ yes > /dev/null &
[2] 5407
[1] Done sleep 100
garima@garima-HP-Pavilion-Notebook:~$ ps
  PID TTY          TIME CMD
 5368 pts/0    00:00:00 bash
 5407 pts/0    00:00:08 yes
 5409 pts/0    00:00:00 ps
garima@garima-HP-Pavilion-Notebook:~$
```

yes, keeps printing "y" again and again. /dev/null throws away all output (so screen doesn't fill up). Runs continuously in background

2. **Basic commands:**

- a) **Ps:**

```
garima@garima-HP-Pavilion-Notebook:~$ ps
  PID TTY          TIME CMD
 5368 pts/0    00:00:00 bash
 5426 pts/0    00:00:00 ps
[2]+ Terminated yes > /dev/null
garima@garima-HP-Pavilion-Notebook:~$
```

shows acurrent running processes

shows all processes with full details

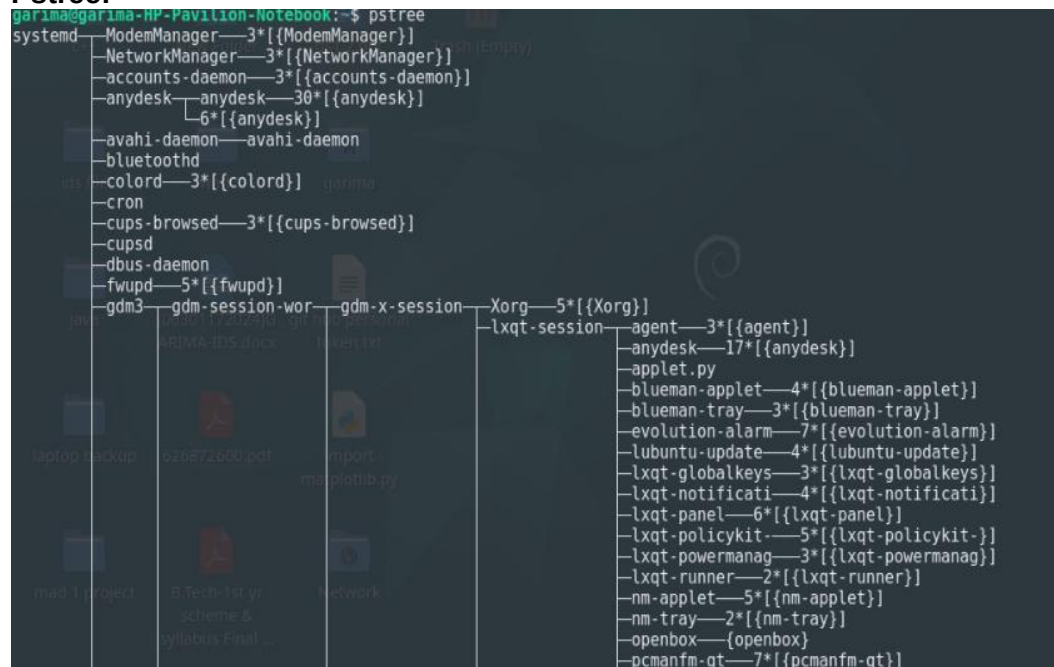
b) Top:

```
top - 21:34:30 up 42 min, 1 user, load average: 0.32, 0.27, 0.48
Tasks: 219 total, 1 running, 218 sleeping, 0 stopped, 0 zombie
%Cpu0 :  4.0 us,  4.0 sy,  0.0 ni, 91.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st  %Cpu1 :  3.1 us,  2.7 sy,  0.0 ni, 93.5 id,  0.0 wa,  0.0 hi,  0.
%Cpu2 :  3.4 us,  3.7 sy,  0.0 ni, 92.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st  %Cpu3 :  3.7 us,  2.7 sy,  0.0 ni, 93.6 id,  0.0 wa,  0.0 hi,  0.
MiB Mem : 7862.7 total, 3296.2 free, 1352.3 used, 3583.7 buff/cache  MiB Swap: 8027.3 total, 8027.3 free, 0.0 used, 6510.4 a
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
5314	garima	20	0	2590896	85896	66956	S	14.0	1.1	1:23.09	anydesk
3935	garima	20	0	922836	154464	110680	S	4.3	1.9	1:03.19	Xorg
1886	root	20	0	572608	33900	29092	S	3.3	0.4	0:20.12	anydesk
3805	garima	9	-11	116216	14204	9584	S	1.0	0.2	0:09.77	pipewire
5364	garima	20	0	589196	102196	86144	S	1.0	1.3	0:00.86	qterminal
1438	message+	20	0	12196	7376	4728	S	0.7	0.1	0:03.21	dbus-daemon
3812	garima	9	-11	115968	17432	10264	S	0.7	0.2	0:06.93	pipewire-pulse

Shows live processes (updates continuously). You can see CPU and memory usage. Press `q` to exit

c) Pstree:



shows processes in a tree format. Helps understand which process started which

d) Lsof:

```
garima@garima-HP-Pavilion-Notebook:~$ lsof
```

COMMAND	PID	TID	TASKCMD	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
systemd	1			root	cwd	unknown				/proc/1/cwd (readlink: Permission denied)
systemd	1			root	rtld	unknown				/proc/1/root (readlink: Permission denied)
systemd	1			root	txt	unknown				/proc/1/exe (readlink: Permission denied)
systemd	1			root	NOFD	unknown				/proc/1/fd (opendir: Permission denied)
kthreadd	2			root	cwd	unknown				/proc/2/cwd (readlink: Permission denied)
kthreadd	2			root	rtld	unknown				/proc/2/root (readlink: Permission denied)
kthreadd	2			root	txt	unknown				/proc/2/exe (readlink: Permission denied)
kthreadd	2			root	NOFD	unknown				/proc/2/fd (opendir: Permission denied)
pool_work	3			root	cwd	unknown				/proc/3/cwd (readlink: Permission denied)
pool_work	3			root	rtld	unknown				/proc/3/root (readlink: Permission denied)
pool_work	3			root	txt	unknown				/proc/3/exe (readlink: Permission denied)

Shows which processes opened which all files. Useful to check files .

🎯 Assignment 2: Signal Lab

1. Start a long-running process.
2. Send:

- SIGINT
- SIGTSTP
- SIGCONT
- SIGKILL

Solutions:

1. long-running process:

```
garima@garima-HP-Pavilion-Notebook:~$ yes > /dev/null &
[2] 5407
[1] Done sleep 100
garima@garima-HP-Pavilion-Notebook:~$ ps
  PID TTY          TIME CMD
 5368 pts/0    00:00:00 bash
 5407 pts/0    00:00:08 yes
 5409 pts/0    00:00:00 ps
garima@garima-HP-Pavilion-Notebook:~$
```

Yes > /dev/null will run till its gets manual terminated or is controlled by a function or a script.

2. Signals and Their Behavior:

Signal	How to Send	What it Does	Behavior Type
SIGINT	Ctrl + C	Stops the process	Normal stop
SIGTSTP	Ctrl + Z	Pauses the process	Temporary stop
SIGCONT	bg / kill -CONT PID	Resumes paused process	Continue execution
SIGKILL	kill -9 PID	Forcefully kills process	Immediate force stop

Easy way to remember

- **SIGINT** → Stop
- **SIGTSTP** → Pause
- **SIGCONT** → Resume
- **SIGKILL** → Kill (forcefully)

🌀 Assignment 3: PATH Hijack Experiment

1. Create a fake command in a custom directory.
2. Modify **PATH**.
3. Override system command behavior.

Solution:

1. Create a fake command

```
mkdir ~/fakebin  
nano ~/fakebin/ls
```

```
garima@garima-HP-Pavilion-Notebook:~$ ls  
2026-01-22-101351_1366x768_scrot.png  2026-01-22-103241_1222x43_scrot.png  2026-01-22-103843_1263x489_scrot.png  level1  Public  
2026-01-22-101355_1366x768_scrot.png  2026-01-22-103324_956x204_scrot.png  AnyDesk  R  
2026-01-22-101356_1366x768_scrot.png  2026-01-22-103441_1037x118_scrot.png  Desktop  mad1-project  snap  
2026-01-22-102100_1366x768_scrot.png  2026-01-22-103642_1218x94_scrot.png  Documents  Music  Templates  
2026-01-22-102128_955x308_scrot.png  2026-01-22-103650_1225x254_scrot.png  Downloads  Pictures  
2026-01-22-103106_903x236_scrot.png  2026-01-22-103743_902x242_scrot.png  'file handling.py'  project  
garima@garima-HP-Pavilion-Notebook:~$ mkdir ~/fakebin  
garima@garima-HP-Pavilion-Notebook:~$ nano ~/fakebin/ls
```

Adding random text in the file "ls":

```
GNU nano 7.2  
echo "Fake ls running"
```

Make it executable:

```
garima@garima-HP-Pavilion-Notebook:~$ chmod +x ~/fakebin/ls
```

2.Modify PATH

```
garima@garima-HP-Pavilion-Notebook:~$ export PATH=~/fakebin:$PATH
```

This puts the fake folder **before** system folders.

3.Override system command

```
garima@garima-HP-Pavilion-Notebook:~$ ls  
Fake ls running  
garima@garima-HP-Pavilion-Notebook:~$ ls  
Fake ls running
```

Instead of real `ls`, the fake one runs

4.Revert back

```
garima@garima-HP-Pavilion-Notebook:~$ export PATH=$(echo $PATH | sed "s|$HOME/fakebin:||")  
garima@garima-HP-Pavilion-Notebook:~$ ls  
2026-01-22-101351_1366x768_scrot.png  2026-01-22-103241_1222x43_scrot.png  2026-01-22-103843_1263x489_scrot.png  AnyDesk  
2026-01-22-101355_1366x768_scrot.png  2026-01-22-103324_956x204_scrot.png  Desktop  mad1-project  snap  
2026-01-22-101356_1366x768_scrot.png  2026-01-22-103441_1037x118_scrot.png  Documents  Music  Templates  
2026-01-22-102100_1366x768_scrot.png  2026-01-22-103642_1218x94_scrot.png  Downloads  Pictures  
2026-01-22-102128_955x308_scrot.png  2026-01-22-103650_1225x254_scrot.png  'file handling.py'  project  
2026-01-22-103106_903x236_scrot.png  2026-01-22-103743_902x242_scrot.png  fakebin
```

or simply restart terminal

5. Why this is dangerous

- System checks commands in **PATH order**
- If fake command comes first → it replaces real one
- Can be misused to:
 - Steal passwords
 - Run harmful scripts
 - Trick users without them knowing